C PROGRAMMING — ASSIGNMENT 1

Task 11: Recommend and Defend the Final Design

Marks: 4 | Total assignment: 50 marks

Recommendation

Recommended design: validate input as text, convert to a bounded integer type, use an unsigned 16-bit register representation for the meter when the platform supports it, compute units with defined modulo-65536 rollover arithmetic, and store every charge and bill total as integer paise in int64_t or long long. Use named tariff constants in one header or module, compile with -Wall -Wextra, and include automated tests for boundaries and rollover.

Limitations

A single rollover formula assumes at most one wrap between readings and cannot distinguish a rollover from an invalid decreasing reading without additional context. A fixed tariff compiled into the program requires rebuilding and redistribution after tariff changes. Integer paise is exact only while calculations remain within the selected integer range. District-scale totals should be checked for overflow.

Configuration file alternative

Loading t tariff values from a configuration file takes O(t) startup time and O(t) memory (or O(1) extra memory if parsed and retained in a fixed structure). It allows tariff revisions without recompilation but requires robust validation, versioning and error handling. A compiled constant table is simpler and catches some mistakes at build time but requires a new build for every revision.

C Source Code / Implementation

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#define FIXED_CHARGE_PAISE INT64_C(5000)
#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE2_PAISE INT64_C(225)
#define RATE3_PAISE INT64_C(450)

static uint32_t units_used(uint16_t prev, uint16_t curr) {
    return (uint16_t)(curr - prev);
}
static int64_t bill_paise(uint32_t u) {
    int64_t total = FIXED_CHARGE_PAISE;
    if (u > SLAB1_UNITS) {
        uint32_t second = (u < SLAB2_UNITS ? u : SLAB2_UNITS) - SLAB1_UNITS;
        total += (int64_t)second * RATE2_PAISE;
    }
    if (u > SLAB2_UNITS) total += (int64_t)(u - SLAB2_UNITS) * RATE3_PAISE;
    return total;
}
int main(void) {
    uint16_t prev = 1000, curr = 1350;
    uint32_t u = units_used(prev, curr);
    int64_t p = bill_paise(u);
    printf("Units=%" PRIu32 ", total=Rs %" PRId64 ".%02" PRId64 "\n",
           u, p / 100, p % 100);
    return 0;
}
```

Submission note

Compile and run the code on your own system, save actual terminal output/screenshots, and include them in your report. Review the code and be prepared to explain it during the viva. Any machine-specific measurements must come from your own execution.